

Indicaciones específicas:

- Esta evaluación contiene 8 páginas.
 - La solución de la tarea deber ser entregada con un link al repositorio en GitHub.
-

Tarea #2: CoreNumeric

Diseño de una librería numérica genérica moderna con C++20

1 Resumen de la tarea

En esta tarea el estudiante desarrollará una **librería numérica genérica**. El enfoque principal es el uso de **Programación Genérica moderna en C++20**, enfatizando:

- Templates
- Concepts
- Variadic templates
- Fold expressions
- Restricciones en tiempo de compilación

El objetivo es comprender cómo se diseñan **APIs genéricas reales**, seguras y expresivas.

Condiciones generales

- Duración: **1 semana**
- Modalidad: **Individual**
- Entrega: enlace a un repositorio GitHub
- Compilación obligatoria con **C++20**
- El archivo `README.md` debe contener únicamente el nombre del estudiante

2 Concepts en C++20

Los *Concepts* permiten restringir plantillas en tiempo de compilación, especificando qué operaciones debe soportar un tipo.

Ejemplo de Concept #1: Concepts para sumar

El concept `Addable` se puede utilizar cuando una función solo necesita sumar valores, sin importar si provienen o no de un contenedor.

```
#include <concepts>

template<typename T>
concept Addable = requires(T a, T b) {
    { a + b } -> std::same_as<T>;
};
```

Uso en una función genérica

```
template<Addable T>
T add(T a, T b) {
    return a + b;
}
```

Casos que compilan

```
add(2, 3);           // OK
add(1.5, 2.5);       // OK
```

Caso que NO compila

```
add("Hola", "Mundo"); // ERROR de compilacion
```

El operador `+` no está definido para `const char*`, por lo que el concept no se cumple.

Ejemplo de Concept #2: Concepts para iterar

El concept `Iterable` se puede utilizar cuando una función solo necesita recorrer un contenedor, sin imponer restricciones sobre el tipo de los elementos.

Definición del concept `Iterable`

```
template<typename C>
concept Iterable = requires(C c) {
    std::begin(c);
    std::end(c);
};
```

Función que solo requiere Iterable

```
template<Iterable C>
std::size_t count_elements(const C& container)
{
    std::size_t count = 0;

    for (const auto& _ : container)
        ++count;

    return count;
}
```

Caso que compila

```
std::vector<int> v{1, 2, 3};
count_elements(v);
```

Caso que no compila

```
int x = 5;
// count_elements(x);
```

El concept `Iterable` restringe **cómo** se **accede a los datos**, no qué operaciones se realizan sobre ellos.

Ejemplo de Concept #3: Iterar objetos con capacidad de sumarse

Cuando una función necesita recorrer un contenedor y operar matemáticamente sobre sus elementos, es necesario combinar concepts mediante la cláusula `requires`.

Función sum

```
template<Iterable C>
requires Addable<typename C::value_type>
auto sum(const C& container)
{
    using T = typename C::value_type;
```

```
T result{}; // Para tipos numericos (int, double, float)
           // se inicializa en 0.
           // Para clases se llama al constructor por defecto
for (const auto& value : container)
    result = result + value;

return result;
}
```

Caso que compila

```
int main(){
    std::vector<int> v{1, 2, 3};
    sum(v);
}
```

Caso que no compila

```
int main(){
    std::vector<std::string> words{"Hola", "Mundo"};
    // sum(words);
}
```

En resumen, **Iterable** controla cómo se recorren los datos y **Addable** controla qué operaciones pueden aplicarse a los datos. La cláusula **requires** combina ambas restricciones y define cuándo la función existe.

Conclusión

El uso correcto de **requires** permite:

- Detectar errores en tiempo de compilación.
- Diseñar funciones genéricas seguras y expresivas.
- Construir APIs similares a las de la STL moderna.

Este patrón será utilizado y evaluado a lo largo de los requerimientos de esta tarea.

Requerimientos de la Tarea

1. Concepts obligatorios y personalizados

El estudiante debe definir y utilizar los siguientes concepts:

Concept	Propósito	Usado en
Addable	Permite suma	sum, variadic, transform
Divisible	Permite división	mean
Iterable	Permite iterar	varios algoritmos

Aquí se muestran como deben ser estos cuatro concepts:

```
#include <concepts>
#include <iterator>

template<typename C>
concept Iterable = requires(C c) {
    std::begin(c);
    std::end(c);
};

template<typename T>
concept Addable = requires(T a, T b) {
    { a + b } -> std::same_as<T>;
};

template<typename T>
concept Divisible = requires(T a, std::size_t n) {
    { a / n } -> std::same_as<T>;
};

// TAREA: Crear uno adicional y aplicarlo
```

Además, el estudiante **debe crear al menos un concept propio desde cero** y aplicarlo en alguna de las siguientes requerimientos:

2. Algoritmo mean

Calcula el promedio del contenedor.

- Debe reutilizar `sum`
- Debe requerir `Divisible`

```
std::vector<double> v{1.0,2.0,3.0};
auto m = core_numeric::mean(v); // usar 'namespace'
                                // para 'core_numeric'
```

3. Algoritmo variance

Calcula la varianza del contenedor.

- Debe reutilizar la función `mean`
- Debe estar restringida mediante los conceptos `Iterable` y `Addable`

Ejemplo de uso en el `main()` (válido):

```
int main() {
    std::vector<double> data{1.0, 2.0, 3.0};

    auto v = core_numeric::variance(data); // Compila
}
```

Ejemplo de uso en el `main()` (inválido):

```
int main() {
    std::vector<std::string> data{"a", "b", "c"};
    auto v = core_numeric::variance(data);
    // ERROR: std::string no cumple los concepts
    // Addable / Comparable requeridos por variance
}
```

4. max

Calcula el máximo elemento de un contenedor.

Ejemplo de uso en el `main()` (válido):

```
int main() {
    std::vector<double> data{1, 2.7, 0.3};

    auto v = core_numeric::max(data); // Compila (debe ser 2.7)
}
```

Ejemplo de uso en el `main()` (inválido):

```
int main() {
    std::vector<std::string> data{"a", "b", "c"};
    auto v = core_numeric::max(data);
    // ERROR: std::string no debe cumplir con el concept
}
```

5. transform_reduce

Aplica una función a cada elemento y reduce el resultado de la siguiente manera:

```
auto r = core_numeric::transform_reduce(v, [](double x){
    return x * x;
});
```

La función debe recibirse como parámetro template.

6. Variadic templates y fold expressions

Implementar una función que acepte múltiples argumentos:

```
auto s1 = core_numeric::sum_variadic(1,2,33,4);
auto s2 = core_numeric::mean_variadic(0.1,2,3,4);
auto s3 = core_numeric::variance_variadic(1,2,3,4);
auto s4 = core_numeric::max_variadic(1,2.7,3,4);
```

Debe usar fold expressions y concepts.

7. Uso obligatorio de if constexpr

Debe diferenciar comportamiento según el tipo:

```
if constexpr (std::is_integral_v<T>) {
    // enteros
} else {
    // flotantes
}
```

No se permite lógica en tiempo de ejecución. Utilizar esta herramienta en alguno de los algoritmos previos.

8. Casos que deben compilar y NO compilar

- El estudiante debe incluir ejemplos en los cuales las llamadas a las funciones compilan correctamente y ejemplos en los cuales no compile el código. Colocar estos ejemplos en un archivo llamado `test.cpp`, un archivo `test.cpp` para cada implementación.
- Utilizar ejemplos con tipos de datos fundamentales (`int`, `double`, `char`, `string`, etc) y también para objetos de clases (crear clases como `Coordenadas`, `Vectores3D`, `Esfera`, etc), utilizar un método para comparar estos objetos.
- Debe explicarse qué concepto falla en cada caso en los comentarios.